

```
#question 1
```

```
L=[10,0,20,40,10,70,0,30,60,80]
print(L)
```

```
[10, 0, 20, 40, 10, 70, 0, 30, 60, 80]
```

```
L=[10,0,20,40,10,70,0,30,60,80]
```

```
L.append(100)
L.insert(2,30)
print(L)
#append adds element at last and insert adds element at specified index
```

```
[10, 0, 30, 20, 40, 10, 70, 0, 30, 60, 80, 100]
```

```
L=[10,0,20,40,10,70,0,30,60,80]
```

```
L.remove(20)
print(L)
L.pop()
print(L)
```

```
[10, 0, 40, 10, 70, 0, 30, 60, 80]
[10, 0, 40, 10, 70, 0, 30, 60]
```

```
L=[10,0,20,40,10,70,0,30,60,80]
```

```
L.sort()
print(L)
L.sort(reverse=True)
print(L)
```

```
[0, 0, 10, 10, 20, 30, 40, 60, 70, 80]
[80, 70, 60, 40, 30, 20, 10, 10, 0, 0]
```

```
L=[10,0,20,40,10,70,0,30,60,80]
```

```
print(L[:3])
print(L[-3:])
```

```
[10, 0, 20]
[30, 60, 80]
```

```
L=[10,0,20,40,10,70,0,30,60,80]
```

```
avg=sum(L)/len(L)
new_list=[x for x in L if x>avg]
print(new_list)
```

```
[40, 70, 60, 80]
```

```
#question 2
```

```
scores=(10,0,20,40,10,70,0,30)
highest=max(scores)
index=scores.index(highest)
lowest = min(scores)
frequency = scores.count(lowest)
```

```
scores=(10,0,20,40,10,70,0,30)
reversed_scores = list(scores[::-1])
print(reversed_scores)
```

```
[30, 0, 70, 10, 40, 20, 0, 10]
```

```
scores=(10,0,20,40,10,70,0,30)
score=int(input("enter a score"))
```

```
for i in scores:
    if i==score:
        print("First occurrence index:", scores.index(score))
    else:
        continue
```

```
enter a score20
First occurrence index: 2
```

```
scores=(10,0,20,40,10,70,0,30)
scores[0] = 100 #error because tuples are immutable while lists are not.
```

```
-----
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_657/2007757916.py in <cell line: 0>()
      1 scores=(10,0,20,40,10,70,0,30)
----> 2 scores[0] = 100

TypeError: 'tuple' object does not support item assignment
```

[Explain error](#)

```
scores=(10,0,20,40,10,70,0,30)
first, second, *remaining = scores

print(first)
print(second)
print(remaining)
```

```
10
0
[20, 40, 10, 70, 0, 30]
```

```
#question 3
import random
random.seed(1024170368)

numbers = [random.randint(100, 900) for _ in range(100)]
print(numbers)
```

```
[346, 273, 672, 885, 619, 302, 273, 314, 774, 696, 871, 300, 458, 850, 323, 314, 889, 133, 302, 355, 389, 738, 351, 479, 297,
```

```
import random
random.seed(1024170368)

numbers = [random.randint(100, 900) for _ in range(100)]

odd_numbers=[x for x in numbers if x%2==1]
print(odd_numbers)
```

```
[273, 885, 619, 273, 871, 323, 889, 133, 355, 389, 351, 479, 297, 341, 287, 143, 269, 709, 177, 811, 549, 229, 583, 483, 655,
```

```
import random
random.seed(1024170368)

numbers = [random.randint(100, 900) for _ in range(100)]

even_numbers=[x for x in numbers if x%2==0]
print(even_numbers)
```

```
[346, 672, 302, 314, 774, 696, 300, 458, 850, 314, 302, 738, 862, 592, 366, 554, 462, 500, 192, 196, 522, 118, 596, 398, 706,
```

```
import random
random.seed(1024170368)

numbers = [random.randint(100, 900) for _ in range(100)]

def is_prime(num):
    if num < 2:
        return False

    for i in range(2, num // 2 + 1):
        if num % i == 0:
            return False

    return True

prime_numbers = [num for num in numbers if is_prime(num)]
prime_count = len(prime_numbers)
print(prime_numbers)
print(prime_count)
```

```
[619, 389, 479, 269, 709, 811, 229, 163, 761, 757]
10
```

```

import random
random.seed(1024170368)

numbers = [random.randint(100, 900) for _ in range(100)]

most_frequent = numbers[0]
highest_count = numbers.count(numbers[0])

for num in numbers:
    count = numbers.count(num)

    if count > highest_count:
        highest_count = count
        most_frequent = num

print("Most frequent number:", most_frequent)
print("Number of times it occurs:", highest_count)

```

```

Most frequent number: 273
Number of times it occurs: 2

```

```

#question 4

digits = [1,0,2,4,1,7,0,3,6,8]

A = {digit * 7 for digit in digits}
B = {digit * 9 for digit in digits}

union = A.union(B)
inter = A.intersection(B)
print(union)
print(inter)

A_minus_B = A.difference(B)

B_minus_A = B.difference(A)
print(A_minus_B)
print(B_minus_A)

#symmetric difference does both a-b and b-a at once.

sym = A.symmetric_difference(B)
print(sym)

print("A subset of B:", A.issubset(B))
print("B superset of A:", B.issuperset(A))
x = int(input("Enter value to remove from a: "))

A.discard(x)
#discard doesnt return an error in case x doesnt exist in A

```

```

{0, 36, 7, 72, 9, 42, 14, 49, 18, 21, 54, 56, 27, 28, 63}
{0}
{7, 42, 14, 49, 21, 56, 28}
{36, 72, 9, 18, 54, 27, 63}
{36, 7, 72, 9, 42, 14, 49, 18, 21, 54, 56, 27, 28, 63}
A subset of B: False
B superset of A: False
Enter value to remove from a: 3

```

```

#question 5

my_dict = {
    "name": "Rahul",
    "roll_no": "1024170368",
    "branch": "CSE",
    "age": 20,
    "city": "Roorkee"
}
dict1 = my_dict.copy()
dict2 = my_dict.copy()

value = my_dict.pop("city")
my_dict["location"] = value
print(my_dict)

my_dict["cgpa"] = 9
my_dict["age"] = my_dict["age"] + 1

removed_branch = dict1.pop("branch")

```

```
print(removed_branch)

del dict2["branch"]

#pop returns the value of the key-value pair deleted whereas del simply deleted without returning anything.
```

```
{'name': 'Rahul', 'roll_no': '1024170368', 'branch': 'CSE', 'age': 20, 'location': 'Roorkee'}
CSE
```

```
my_dict = {
    "name": "Rahul",
    "roll_no": "1024170368",
    "branch": "CSE",
    "age": 20,
    "city": "Roorkee"
}
for key, value in my_dict.items():
    print(key, "→", value)

if "email" in my_dict:
    print("Email:", my_dict["email"])
else:
    print("Email is not present.")

friend_dict = {
    "name": "Aman",
    "roll_no": "9876543210",
    "branch": "ECE",
    "age": 21,
    "city": "Delhi"
}

merged = {**my_dict, **friend_dict}

print(merged)
# When both dictionaries have the same key, the value from the second dictionary wins.

string_values = {
    key: value
    for key, value in my_dict.items()
    if isinstance(value, str)
}

print("Only string values:", string_values)
```

```
name → Rahul
roll_no → 1024170368
branch → CSE
age → 20
city → Roorkee
Email is not present.
{'name': 'Aman', 'roll_no': '9876543210', 'branch': 'ECE', 'age': 21, 'city': 'Delhi'}
Only string values: {'name': 'Rahul', 'roll_no': '1024170368', 'branch': 'CSE', 'city': 'Roorkee'}
```

Start coding or [generate](#) with AI.